AI-Powered Personalized Learning Path Recommender

Complete React.js / Next.js Frontend Architecture & Implementation Document

Purpose: This document consolidates the frontend architecture derived from the project functionality and the system architecture supplied in the conversation. It is intended as the frontend implementation blueprint for the HCLTech hackathon.

Architecture source of truth

Frontend → Scala Backend → Python/FastAPI + Agno AI Service → PostgreSQL/pgvector, with Redis where useful. The frontend must never directly depend on the AI service. Scala owns application state and business logic; Python/Agno owns AI reasoning.

1. Product Understanding

The platform is not a simple course recommender. It understands the learner's goal, interests, career aspirations, experience, learning preferences, available time, progress, skill confidence and assessment signals, then creates and continuously adapts a personalized learning path.

```
Learner Goal
   ↓
Learner Profile
   ↓
Skill / Prerequisite Graph
   ↓
Personalized Learning Path
   ↓
Resource Discovery
   ↓
Resource Analysis
   ↓
Recommendation / Ranking
   ↓
Learning
   ↓
Assessment / Feedback
   ↓
Learner State Update
   ↓
Path Adaptation
   ■
```

The frontend's job is to make this intelligence understandable and actionable: collect learner input, visualize the resulting path, expose resources, track progress, show adaptation, and provide a contextual assistant.

2. Frontend Responsibilities

• Landing/home experience and product explanation.

• Authentication UI using Clerk or OAuth integration through the application architecture.

• Onboarding questionnaire and learner-profile setup.

• RIASEC career-interest assessment for eligible class 10–12 students.

• Learning-goal creation and editing.

• Personalized learning-path visualization with prerequisites, phases, nodes and milestones.

• Resource/course/video/article/project discovery and detail pages.

• Resource recommendations and ranking presentation.

• Progress, skills, milestones and assessment interfaces.

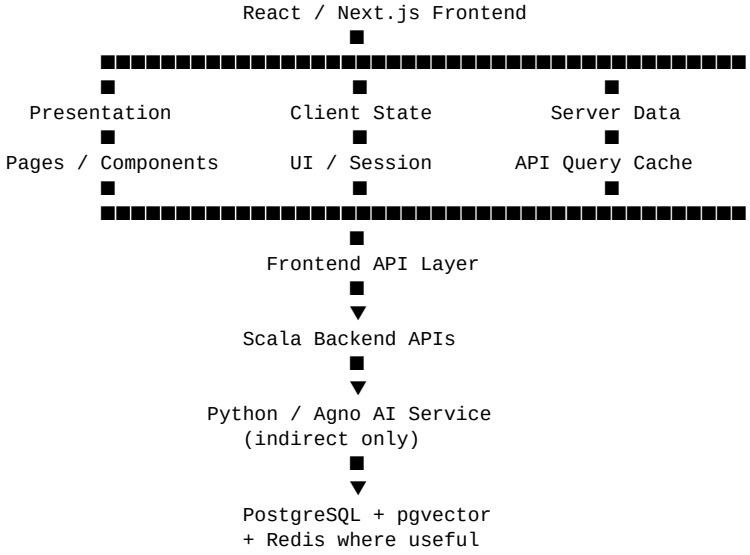
• Learner activity/event capture through backend APIs.

• Adaptive roadmap and recommendation changes.

• Contextual conversational AI assistant using the current learner state.

• Responsive, accessible, modern learning-platform UX.

3. End-to-End Frontend Architecture



Critical rule: React calls only the Scala backend. AI endpoints such as /ai/generate-learning-path are backend-to-AI contracts, not browser-facing dependencies.

4. Recommended Frontend Stack

• **Framework:** Next.js App Router + TypeScript (React architecture remains applicable if the team chooses plain React + React Router).

• **Styling:** Tailwind CSS.

• **UI primitives:** shadcn/ui or an equivalent accessible component system.

• **Icons:** Lucide React.

• **Animations:** Framer Motion, used selectively for transitions, roadmap interactions and feedback states.

• **Server data:** TanStack Query for API fetching, caching, mutations and invalidation.

• **Client UI state:** Zustand or Context for small cross-page UI state; do not duplicate server state there.

• **Forms:** React Hook Form + Zod.

• **Charts:** Recharts or equivalent for progress/skill analytics.

• **Graph visualization:** React Flow or a custom SVG/canvas visualization for the skill/prerequisite graph.

• **Authentication:** Clerk/OAuth, with backend authorization remaining authoritative.

• **Testing:** Vitest/Jest + React Testing Library; Playwright for end-to-end flows.

5. Frontend Project Structure

```
src/
■■■ app/
■   ■■■ (marketing)/
■   ■   ■■■ page.tsx
```

```
■   ■   ■■■ features/page.tsx
■   ■■■ (auth)/
■   ■   ■■■ sign-in/page.tsx
■   ■   ■■■ sign-up/page.tsx
■   ■■■ onboarding/
■   ■   ■■■ page.tsx
■   ■   ■■■ goals/page.tsx
■   ■   ■■■ interests/page.tsx
■   ■   ■■■ experience/page.tsx
■   ■   ■■■ preferences/page.tsx
■   ■   ■■■ availability/page.tsx
■   ■   ■■■ riasec/page.tsx
■   ■■■ dashboard/
■   ■   ■■■ page.tsx
■   ■   ■■■ layout.tsx
■   ■   ■■■ learning-path/page.tsx
■   ■   ■■■ learning-path/[pathId]/page.tsx
■   ■   ■■■ resources/page.tsx
■   ■   ■■■ resources/[resourceId]/page.tsx
■   ■   ■■■ assessments/page.tsx
■   ■   ■■■ assessments/[assessmentId]/page.tsx
■   ■   ■■■ progress/page.tsx
■   ■   ■■■ skills/page.tsx
■   ■   ■■■ milestones/page.tsx
■   ■   ■■■ goals/page.tsx
■   ■   ■■■ assistant/page.tsx
■   ■■■ loading.tsx / error.tsx / not-found.tsx
■
■■■ components/
■   ■■■ ui/
■   ■■■ layout/
■   ■■■ landing/
■   ■■■ auth/
■   ■■■ onboarding/
■   ■■■ riasec/
■   ■■■ dashboard/
■   ■■■ learning-path/
■   ■■■ skill-graph/
■   ■■■ resources/
■   ■■■ assessments/
■   ■■■ progress/
■   ■■■ milestones/
■   ■■■ assistant/
■
■■■ features/
■   ■■■ profile/
■   ■■■ goals/
■   ■■■ learning-path/
■   ■■■ resources/
■   ■■■ progress/
■   ■■■ assessments/
■   ■■■ recommendations/
■   ■■■ riasec/
■   ■■■ assistant/
■
■■■ lib/
■   ■■■ api/
■   ■   ■■■ client.ts
■   ■   ■■■ profile.ts
■   ■   ■■■ goals.ts
■   ■   ■■■ learning-paths.ts
■   ■   ■■■ resources.ts
■   ■   ■■■ progress.ts
■   ■   ■■■ assessments.ts
■   ■   ■■■ recommendations.ts
■   ■   ■■■ assistant.ts
■   ■■■ auth/
■   ■■■ query/
■   ■■■ validation/
■   ■■■ analytics/
■   ■■■ utils/
■
■■■ hooks/
■■■ stores/
■■■ types/
■   ■■■ learner.ts
■   ■■■ goal.ts
■   ■■■ skill.ts
■   ■■■ learning-path.ts
```

```
■   ■■■ resource.ts
■   ■■■ progress.ts
■   ■■■ assessment.ts
■   ■■■ recommendation.ts
■   ■■■ assistant.ts
■
■■■ constants/
```

6. Architecture Layers Inside the Frontend

```
UI Layer
  ↓
Feature Components
  ↓
Feature Hooks / View Models
  ↓
TanStack Query / Client State
  ↓
Typed API Client
  ↓
Scala Backend

Types / Zod schemas sit beside the API layer and validate external data.

UI components should not know endpoint URLs.
Pages should compose features rather than contain business logic.
Server state should live in the query cache.
Client-only state should remain small and local.
```

7. Main Route Map

Public: /, /features, /about (optional), /sign-in, /sign-up

Onboarding: /onboarding → /onboarding/goals → /onboarding/interests → /onboarding/experience → /onboarding/preferences → /onboarding/availability → /onboarding/riasec (conditional) → /onboarding/review

Application: /dashboard, /dashboard/learning-path, /dashboard/learning-path/[pathId], /dashboard/resources, /dashboard/resources/[resourceId], /dashboard/assessments, /dashboard/assessments/[assessmentId], /dashboard/progress, /dashboard/skills, /dashboard/milestones, /dashboard/goals, /dashboard/assistant

8. Landing + Authentication

```
Landing
  ■■■ Hero
  ■■■ How it works
  ■■■ Personalization explanation
  ■■■ Adaptive-learning explanation
  ■■■ Feature highlights
  ■■■ CTA
          ↓
     Sign in / Sign up
          ↓
  Existing profile? ■■ Yes ■■→ Dashboard
          ■
          No
          ↓
       Onboarding
```

Authentication state should be resolved before protected routes render. The backend remains authoritative for user identity, authorization and persistent profile state.

9. Onboarding Architecture

```
Onboarding Shell
  ■■■ Progress indicator
  ■■■ Back / Next
  ■■■ Save draft
  ■■■ Exit / Resume

Step 1: Goals
Step 2: Interests
Step 3: Career aspirations
```

```
Step 4: Experience
Step 5: Learning depth
Step 6: Learning preferences
Step 7: Available time
Step 8: RIASEC (only when applicable)
Step 9: Review & confirm
                        ↓
                PUT /profile
                POST /learning-goals
                        ↓
                Profile Ready
```

Use React Hook Form + Zod. Keep a single typed onboarding draft model. Persist partial progress when appropriate so a learner can resume.

10. RIASEC Assessment

For learners in classes 10–12, the frontend should show the RIASEC assessment as a dedicated, distraction-free flow. The UI collects answers, shows completion progress and submits the result through the Scala backend.

```
RIASEC
  ↓
Question cards
  ↓
Answer selection
  ↓
Progress indicator
  ↓
Review (optional)
  ↓
POST /assessments or profile/RIASEC endpoint
  ↓
RIASECResults persisted by backend
  ↓
Career exploration / goal suggestions
  ↓
Learner confirms goal
```

11. Learner Profile UI

The profile screen should clearly separate information that changes slowly from dynamic personalization signals.

```
Learner Profile
■■■ Static
■   ■■■ Experience
■   ■■■ Goals
■   ■■■ Career objectives
■   ■■■ Interests
■   ■■■ Learning preferences
■   ■■■ Available time
■   ■■■ Completed resources
■   ■■■ RIASEC results
■
■■■ Dynamic
    ■■■ Skill confidence
    ■■■ Completion rate
    ■■■ Behaviour signals
    ■■■ Resource preferences
    ■■■ Learning-style signals
    ■■■ Assessment performance
    ■■■ Strengths / weaknesses
    ■■■ Current roadmap position
```

12. Learning Goal UI

Provide a goal-management experience where learners can view the active goal, target role, timeframe and related skills. Creating or changing a goal should go through Scala APIs; AI may assist with interpretation but does not own the persisted goal.

```
Goal selected
   ↓
POST /learning-goals
   ↓
Scala persists goal
```

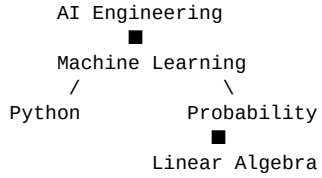
```
   ↓
Generate path action
   ↓
POST /learning-paths (application API)
   ↓
Scala calls Python /ai/generate-learning-path
   ↓
Persist generated path
   ↓
Dashboard refresh
```

13. Learning Path Experience

```
Dashboard
   ■
   ■■■ Current phase
   ■■■ Next recommended action
   ■■■ Overall progress
   ■■■ Skill progress
   ■■■ Upcoming milestone
   ■■■ Continue learning
            ↓
Learning Path
   ■■■ Phase 1
   ■    ■■■ Skill node
   ■    ■■■ Resource
   ■    ■■■ Assessment
   ■■■ Phase 2
   ■■■ Phase 3
            ↓
Prerequisite / dependency visualization
```

Recommended components: PathHeader, PathProgress, PhaseSection, LearningPathNode, SkillBadge, PrerequisiteList, MilestoneCard, ContinueButton, LockedNode, CompletedNode, RecommendedNode.

14. Skill & Prerequisite Graph



The frontend visualizes relationships returned by Scala. Core prerequisite validation and unlocking remain deterministic backend rules. React should never decide that a learner is allowed to unlock a skill merely because an AI response says so.

15. Resource Discovery & Recommendation UI

```
Learning Path / Skill
        ↓
Recommended Resources
        ↓
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Resource Card                    ■
■ title                            ■
■ provider                         ■
■ type                             ■
■ duration                         ■
■ difficulty                       ■
■ skills covered                   ■
■ match / relevance indicators     ■
■ learner preference fit           ■
■ progress                         ■
■ Save / Start / Complete          ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Resource types include courses, YouTube videos, projects, articles and other learning resources. Resource metadata and ranking are AI-enriched, but the frontend simply consumes the structured recommendation response from Scala.

16. Resource Detail Page

• Title, provider and content type.

• Description and topic.

• Skills covered.

• Difficulty and depth.

• Duration.

• Prerequisites.

• Quality/freshness signals where exposed by backend.

• Why this resource was recommended.

• Progress state.

• Start/open resource.

• Save resource.

• Mark complete.

• Feedback action for future personalization.

17. Dashboard Architecture

```
Dashboard
■■■ Greeting / learner context
■■■ Current goal
■■■ Continue learning
■■■ Learning path progress
■■■ Current phase
■■■ Recommended next action
■■■ Skill progress
■■■ Upcoming milestone
■■■ Recommended resources
■■■ Recent activity
■■■ Assessment snapshot
■■■ AI Assistant entry point
```

The dashboard should feel like a modern learning platform while exposing the unique differentiator: the system continuously explains what the learner should do next and why.

18. Progress Tracking

```
User action
   ↓
Frontend records semantic event
   ↓
POST /progress/events
   ↓
Scala validates + persists event
   ↓
Learner state update
   ↓
Recommendations/path may change
   ↓
React invalidates affected queries
   ↓
Dashboard reflects new state
```

Examples: RESOURCE_STARTED, RESOURCE_COMPLETED, ASSESSMENT_SUBMITTED, SKILL_CONFIDENCE_UPDATED, MILESTONE_COMPLETED, RESOURCE_SAVED, RESOURCE_SKIPPED.

19. Assessment UI

```
Assessment list
   ↓
```

```
Assessment detail
   ↓
Question navigation
   ↓
Answer validation / local draft
   ↓
Submit
   ↓
POST /assessments/{id}/submit
   ↓
Scala persists result
   ↓
Learner state changes
   ↓
Recommendations refresh
   ↓
Possible path adaptation
```

Do not calculate authoritative assessment results only on the client. The backend should own submission and result persistence.

20. Milestones

Milestones are displayed as progress checkpoints. The frontend renders locked, active, completed and upcoming states based on backend state.

```
Milestone
   ■■■ title
   ■■■ description
   ■■■ target
   ■■■ progress
   ■■■ status
   ■■■ related learning-path nodes
```

21. Recommendations Experience

```
Recommendation Feed
■■■ Why recommended
■■■ Skill relevance
■■■ Goal alignment
■■■ Difficulty match
■■■ Preference fit
■■■ Duration fit
■■■ Quality
■■■ Freshness
■■■ Current progress
■■■ Actions: Start / Save / Dismiss / Complete
```

The frontend should show explainability signals when the backend exposes them. This is particularly valuable for an AI-powered system because the learner should understand why a recommendation appeared.

22. Adaptive Learning Path UX

```
Learner activity / assessment
          ↓
      Backend event
          ↓
    LearnerState update
          ↓
    AI adaptation request
          ↓
   Adapted path returned
          ↓
Scala validates + persists
          ↓
Frontend receives updated path
          ↓
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Path updated                  ■
■ • New prerequisite module   ■
■ • Reordered skill             ■
■ • New resource                ■
■ • Milestone changed           ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Show an unobtrusive 'Your path was updated' explanation. Never silently make confusing changes. If possible, expose what changed and why.

23. Conversational AI Assistant

```
Assistant UI
   ↓
POST /assistant/chat
   ↓
Scala gathers current LearnerState
   ↓
Scala → Python /ai/chat
   ↓
Agno reasons over learner context
   ↓
Structured/streamed response
   ↓
Scala returns response
   ↓
Frontend renders answer
```

The assistant is not a separate chatbot persona. It should understand the same goal, roadmap, progress, skills, weaknesses and recommendations represented by LearnerState.

UI should support contextual prompts such as: 'Why is this next?', 'What should I learn this week?', 'I am struggling with this skill', 'Can you explain this resource?', and 'What changed in my roadmap?'.

24. API Client Architecture

```
lib/api/client.ts
  ■■■ authenticatedFetch()
  ■■■ error normalization
  ■■■ request IDs
  ■■■ common headers

lib/api/profile.ts
lib/api/goals.ts
lib/api/learning-paths.ts
lib/api/resources.ts
lib/api/progress.ts
lib/api/assessments.ts
lib/api/recommendations.ts
lib/api/assistant.ts

Components → hooks → API modules → Scala backend
```

Keep endpoint details out of UI components. Each API module should expose typed functions and return typed data. Use a consistent ApiError shape.

25. Suggested Query/Mutation Hooks

```
useLearnerProfile()
useUpdateLearnerProfile()

useLearningGoals()
useCreateLearningGoal()
useUpdateLearningGoal()

useLearningPaths()
useLearningPath(pathId)
useGenerateLearningPath()

useResources(filters)
useResource(resourceId)
useRecommendedResources()

useProgress()
useRecordProgressEvent()

useAssessments()
useAssessment(id)
useSubmitAssessment()
```

```
useSkills()
useMilestones()

useRecommendations()
useAssistantChat()
```

26. State Management Strategy

```
SERVER STATE
TanStack Query
  → profile, goals, path, resources, progress, assessments,
    recommendations, milestones, assistant history

CLIENT/UI STATE
React local state / Zustand
  → modal state, sidebar, filters, onboarding draft,
    temporary selections, UI preferences

FORM STATE
React Hook Form
  → onboarding, profile edits, assessments

DO NOT
put the entire backend database model into a global Zustand store.
```

Use query invalidation after mutations rather than manually maintaining multiple copies of server state.

27. Type and Schema Strategy

Define frontend TypeScript types that mirror the agreed Scala API contracts. Use Zod at the boundary for runtime validation of important responses.

```
type LearnerState = {
  profile: LearnerProfile;
  goals: LearningGoal[];
  skills: SkillProgress[];
  currentPath?: LearningPath;
  milestones: MilestoneProgress[];
  recentActivity: ProgressEvent[];
  recommendations: Recommendation[];
};

type LearningPath = {
  id: string;
  goalId: string;
  title: string;
  description?: string;
  status: "draft" | "active" | "completed" | "adapted";
  phases: LearningPathPhase[];
  overallProgress: number;
};

type LearningPathNode = {
  id: string;
  type: "skill" | "resource" | "assessment" | "milestone";
  title: string;
  status: "locked" | "available" | "in_progress" | "completed";
  prerequisites: string[];
};
```

28. Frontend-to-Backend Contract Examples

```
GET  /profile
PUT  /profile

POST /learning-goals
GET  /dashboard

POST /learning-paths
GET  /learning-paths/{id}

POST /progress/events

POST /assessments/{id}/submit
```

```
GET  /recommendations

POST /assistant/chat
```

The browser should call these application APIs. Scala internally communicates with Python/Agno using the separate AI contracts:

```
POST /ai/generate-learning-path
POST /ai/recommend-resources
POST /ai/adapt-learning-path
POST /ai/chat
POST /ai/generate-assessment
```

29. Error, Loading and Empty States

• Skeleton loading for dashboard/path/resource lists.

• Inline form errors with field-level validation.

• Retry UI for transient API failures.

• Clear empty state when no recommendations exist yet.

• Generation state for a new learning path.

• AI timeout state that never leaves the learner stuck.

• Partial data handling when one dashboard widget fails.

• Unauthorized/expired session handling.

• 404 state for missing paths/resources.

• Adaptation-in-progress state after learner events.

30. AI Generation UX

```
User clicks "Generate my path"
          ↓
Frontend mutation
          ↓
Backend request
          ↓
AI processing
          ↓
Frontend shows progress state:
  • Understanding your goal
  • Mapping required skills
  • Checking prerequisites
  • Building your path
          ↓
Success → Path screen
Failure → Retry / fallback message
```

These progress messages are UX states, not claims about the exact internal agent execution unless the backend explicitly provides those stages.

31. Security Boundaries

• Never expose AI service credentials or provider API keys in the browser.

• Never trust client-side user IDs for authorization.

• Use the authenticated session/token to call Scala.

• Backend validates ownership of profile, path, progress and assessment resources.

• Treat AI output as untrusted structured data and validate it before rendering or persisting.

• Sanitize/render rich resource descriptions safely.

• Avoid storing unnecessary sensitive learner data in client storage.

• Do not put PostgreSQL credentials, Redis credentials or AI keys into frontend environment variables.

32. Event-Driven Frontend Pattern

```
UI Action
  ↓
Semantic event builder
  ↓
recordProgressEvent()
  ↓
Scala
  ↓
Persist ProgressEvent / UserEvent
  ↓
Update LearnerState
  ↓
Recalculate / adapt if required
  ↓
Invalidate:
  profile
  dashboard
  path
  recommendations
  progress
  milestones
```

The frontend should capture meaningful events rather than sending arbitrary UI telemetry as the only source of truth.

33. Complete MVP Vertical Slice

```
AUTH
 ↓
ONBOARDING
 ↓
LEARNER PROFILE
 ↓
ONE LEARNING GOAL
 ↓
Scala application API
 ↓
Python / Agno learning-path generation
 ↓
Structured path
 ↓
Scala validation + persistence
 ↓
Prerequisites + milestones
 ↓
Resource discovery
 ↓
Resource ranking
 ↓
Scala persistence
 ↓
DASHBOARD
 ↓
LEARNER SEES PERSONALIZED PATH
```

This should be the first frontend milestone. Do not build every screen before this vertical slice works end-to-end.

34. Phase 2 Frontend Expansion

```
MVP
 ↓
Progress tracking
 ↓
Resource completion
 ↓
Assessments
 ↓
Skill confidence
 ↓
Learner-state updates
 ↓
```

```
Adaptive recommendations
 ↓
Path adaptation UI
 ↓
Conversational assistant
 ↓
Advanced analytics / RIASEC career exploration
```

35. Component Architecture by Feature

Onboarding: OnboardingShell, StepIndicator, GoalSelector, InterestSelector, CareerGoalForm, ExperienceSelector, LearningDepthSelector, PreferenceSelector, AvailabilitySelector, RIASECQuiz, OnboardingReview.

Learning Path: PathHeader, PathProgress, PhaseSection, PathNode, NodeStatus, PrerequisiteGraph, SkillProgress, MilestoneCard, PathChangeBanner.

Resources: ResourceCard, ResourceGrid, ResourceFilters, ResourceTypeBadge, ResourceMetadata, RecommendationReason, ResourceDetail, SaveResourceButton, ResourceProgress.

Progress: ProgressOverview, SkillProgressChart, ActivityTimeline, AssessmentSummary, MilestoneTimeline, WeeklyGoalCard.

Assistant: AssistantPanel, MessageList, MessageBubble, Composer, SuggestedPrompt, ContextBadge, Typing/StreamingState.

36. Dashboard Data Composition

```
Dashboard page
   ■■■ useLearnerProfile()
   ■■■ useLearningGoals()
   ■■■ useCurrentLearningPath()
   ■■■ useRecommendedResources()
   ■■■ useProgress()
   ■■■ useMilestones()

                ↓

          Dashboard View Model

                ↓

      Independent dashboard widgets

A failure in recommendations should not blank the entire dashboard.
```

37. Responsive UX

• Desktop: persistent sidebar + content workspace + optional assistant panel.

• Tablet: collapsible navigation and two-column dashboard where space permits.

• Mobile: bottom navigation or compact navigation, single-column learning path and resource cards.

• Skill graph should offer a simplified mobile list/step view rather than forcing a dense graph.

• Assistant should become a full-screen sheet on mobile.

38. Suggested Dashboard Layout

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Header: Goal / Search / Notifications / Profile     ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Sidebar        ■ Continue learning                  ■
■                ■ Current roadmap phase              ■
■ Dashboard      ■ Progress                           ■
■ Learning Path  ■ Recommended next action            ■
■ Resources      ■                                     ■
■ Assessments    ■ Recommended resources              ■
■ Progress       ■ Skill progress                     ■
■ Skills         ■ Milestones                         ■
■ Milestones     ■ Recent activity                   ■
```

```
■ Assistant    ■                                     ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

39. Frontend Folder Ownership Rules

- app/: routing, layouts and page composition.
- components/: reusable presentation components.
- features/: feature-specific UI + hooks + view models.
- lib/api/: only typed HTTP communication with Scala.
- hooks/: cross-feature reusable hooks.
- stores/: minimal client/UI state.
- types/: shared TypeScript contract types.
- validation/: Zod schemas.
- Do not place API calls directly inside deeply nested UI components.
- Do not put business rules such as prerequisite unlocking inside React.

40. Development Sequence for the Frontend Team

```
STEP 1
Define API contracts + TypeScript/Zod models
        ↓
STEP 2
Create app shell, auth and route protection
        ↓
STEP 3
Build onboarding with mocked backend responses
        ↓
STEP 4
Build dashboard shell
        ↓
STEP 5
Build learning-path visualization from mocked JSON
        ↓
STEP 6
Build resource cards/detail/ranking UI
        ↓
STEP 7
Connect profile + goals + path APIs
        ↓
STEP 8
Connect real Scala AI-generation workflow
        ↓
STEP 9
Add progress + event capture
        ↓
STEP 10
Add assessments + learner-state refresh
        ↓
STEP 11
Add adaptation UX
        ↓
STEP 12
Add contextual assistant
        ↓
STEP 13
Testing, accessibility, responsive polish
```

41. What the Frontend Should NOT Own

- AI agent orchestration.
- LLM provider calls.
- Skill-gap reasoning.

• Authoritative prerequisite validation.

• Milestone unlocking rules.

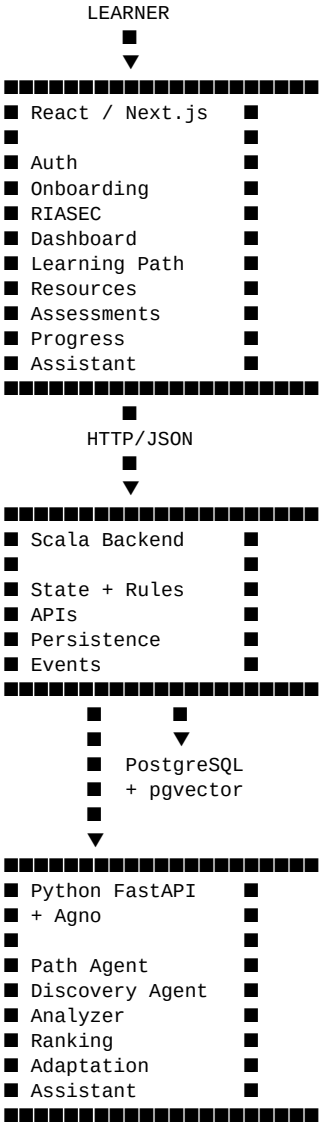
• Persistent learner-state mutation without backend validation.

• Recommendation ranking logic as the source of truth.

• Database access.

• Secrets/API keys.

42. Architecture Summary



43. Final Frontend Architecture Principle

The React application is the learner experience layer, not the intelligence or source-of-truth layer. It should collect structured learner input, render authoritative application state, communicate only with Scala, expose AI-driven recommendations in an understandable way, capture meaningful learner actions, and refresh itself when learner state changes.

The most important implementation discipline is to keep the boundary clean: **React renders and interacts → Scala owns state/rules → Agno reasons → Scala validates/persists → React renders the new state.**

44. API ↔ Frontend Screen Mapping

Backend API	Primary frontend consumer	Purpose
GET/PUT /profile	Profile + onboarding	Read/update learner profile
POST /learning-goals	Goal setup	Create learner goal
GET /dashboard	Dashboard	Compose learner dashboard state
POST /learning-paths	Path generation action	Request application-level path generation
GET /learning-paths/{id}	Learning Path	Render persisted roadmap
POST /progress/events	Learning/resource UI	Capture semantic progress events
POST /assessments/{id}/submit	Assessment UI	Submit assessment
GET /recommendations	Dashboard/resources	Render ranked recommendations
POST /assistant/chat	Assistant	Contextual learner conversation

45. Immediate Frontend Deliverables

• Shared TypeScript interfaces and Zod schemas for LearnerProfile, LearnerState, LearningGoal, Skill, SkillProgress, LearningPath, LearningPathNode, Resource, Recommendation, Assessment, Milestone and ProgressEvent.

• Typed API client for Scala.

• Application shell + authenticated routes.

• Onboarding flow with draft persistence.

• RIASEC flow with conditional routing.

• Dashboard shell.

• Learning-path visualization using mock JSON first.

• Resource recommendation UI using mock JSON first.

• Progress/event utility.

• A single end-to-end MVP flow connected to the real Scala backend.

This document should be used as the frontend architecture baseline. If a later requirement conflicts with it, the change should be explicit and its tradeoffs documented before implementation.